

## Group Project 1: Having a BLAST

- due Friday May 22nd by class time
- 20 pts (see rubric on final page)
- to be submitted with peer reviews (4 of the 20 pts)

## Objectives

- To work in a team comprised of both CS and Biology colleagues to implement a tool that has usefulness in a biological context
- To implement an *alignment matcher*
- To practice with *scoring matrices*
- To think further about algorithm design
- To participate and communicate well in a team

## Introduction

In this project, where you will be assigned to a team, your goal will be to work together to implement a **pairwise alignment matcher**, similar in nature to BLAST. Biology students have already implemented this previously, but in an intractable way that does not scale to real-world problems. You will rectify that deficiency in this project.

## Team Dynamics

You will get the most out of this project if you put effort into working as a team: communicating with your colleagues early and often. There are parts of this project that are really only understandable in-depth by the biologists, and parts that are much more easily grasped by those who have previously taken a computer programming course. That's deliberate. Each of you now has a basic understanding of the other's field, and you should be helping each other understand new concepts. The more you do that, the more effective everyone on the team will be in contributing to a complete project.

## Your Mission

You are to write a program that takes two protein sequences as input (hard coded is ok) and outputs the best alignment using the Needleman-Wunsch algorithm. You should use a gap penalty of -11, which is the gap penalty that BLAST uses by default. In addition, your program should use the BLOSUM 62 scoring matrix to determine the match/mismatch score between any two amino acids. To help you out, I have already written a function that creates a BLOSUM 62 matrix for you. You just have to import it (or copy and paste it) into your project and then use it.

## The Details

To help organize the algorithm, here is a list of functions that would be helpful to write. You are not required to use them, and if you would rather blaze your own trail, feel free.

The **computer science person(s)** on your team will probably be the one to write the bulk of this code, but, in actuality, everyone should be able to contribute in writing parts of this. Be sure you understand how the algorithm works!

```
1 def makeMatrix(rows,cols):
2     # This function creates an empty matrix and returns it.
3     # The parameters tell you what the size of the matrix
4     # should be (number of rows and number of columns).
5     #
6     # The easiest way Python has for creating a matrix
7     # is to use a list of lists. This can be used to
8     # create a matrix for either the scores or the path.
```

```
1 def primeAlignMatrix(scoresMatrix):
2     # Initializes the first row and first column of the
3     # scores matrix. Remember, position [0][0] is initialized
4     # to zero, and the others are computed by repeatedly
5     # applying the gap penalty.
```

```
1 def primePathMatrix(pathMatrix):
2     # If you're using a second matrix to keep track of the
3     # path, this function initializes that matrix. The top
4     # row should always indicate a path heading left, and
5     # the leftmost column should indicate a path moving up.
6     #
7     # You can store whatever you want to indicate the path.
8     # For example, you could use single characters
9     # 'T', 'L', and 'D' to indicate top, left, or diagonal
10    # movement, respectively.
```

```
1 def fill_in_matrix(scoresMatrix, pathMatrix):
2     # After the two matrices have been initialized, this
3     # function fills in the rest of both matrices using
4     # the Needleman and Wunsch algorithm.
5     #
6     # The BLOSUM 62 matrix will be used here.
```

The following helper functions work to print intermediate results of your algorithm. They are highly suggest to help with debugging and ensure your final functions are working as expected and are semantically correct.

```

1 def similar(aa1,aa2):
2     # This function takes two amino acids as parameters and
3     # returns True if they are similar, False if they're not.
4     # Two amino acids are similar if they both belong to the
5     # same group. Here are the groups:
6     #
7     # Negative Polar: D, E
8     # Positive Polar: R, K, H
9     # Uncharged Polar: N, Q, S, T, Y
10    # Non-Polar: A, G, V, L, I, P, F, M, W, C
11    #
12    # You will need to use this when you print the alignment
13    # (see next function) so you know where to put the + symbols
14    # in the consensus line, just like BLAST does.

```

```

1 def printAlignment(pathMatrix):
2     # After the matrices have been filled in, this function
3     # follows the path starting from the lower-right corner of
4     # the path matrix, and prints out the best alignment.
5     #
6     # Remember, diagonal movement indicates an
7     # amino-acid-to-amino-acid alignment while horizontal
8     # or vertical movement indicates the alignment of an
9     # amino acid to a gap. This function prints the
10    # answer. Nothing is returned. When printing
11    # the alignment, you should format it the way BLAST does,
12    # as shown in this example:
13    #
14    # Query  PEFLDHVTKVMLVIDAAVSHLDNLHTLEDFLNLGKKKHQAVGVNTQSFVAVGESLLYMLQ
15    #        P + H+ KV++++ ++ HLDNL+      L L  + V + + F ++G L+ +L
16    # Sbjct  PKVKAHGKKVLGAFSDGLAHLNLTGTFATLSELHCDKLHVDPEN--FRLLGNVLCVLA
17    #
18    # Query  CSLGQGYTAPLRQAWLNMYTIVVAAMSRGWAKNGEHKTD
19    #        +G   T+P+ QA+   Y+ VVA      G+A  + HK+
20    # Sbjct  HHFGKEFTPPV-QAA---YQKVVA----GVANALAHKY-
21    #
22    # where one sequence is shown as the subject, the other is shown
23    # as the query (it doesn't matter which is which) and the consensus
24    # line is shown in the middle. The amino acid is shown for exact
25    # matches, a + is shown for similar, but not exact, matches (using
26    # the similar function above), and a space is shown for mismatches.
27    #
28    # For ease of reading, you should break the alignment up to show
29    # 60 residues per line, and then continue it on the next line, as
30    # shown above.

```

```
1 def printMatrix(someMatrix):
2     # While not necessary for the algorithm to work, squashing
3     # bugs will be a lot easier if you can print the contents
4     # of the matrices easily. That's what this function does.
5     # The parameter is either the scores matrix or the path
6     # matrix, depending on what you want to print.
7     # Hint: print tabs by doing
8     # print "\t",
9     # and move to a new line by doing
10    # print "\n"
11    # This will make it easier to line things up in rows and columns.
```

## Testing

One initial test you should try is to give it the EMSY protein that failed miserably in the straightforward version of BLAST. Align EMSY against a 3-amino-acid protein and see if you get an answer in a shorter amount of time. Then try pairs of sequences where each has a length longer than 3.

The input for your next set of tests will be some human proteins that you have researched as a group. Start by canvassing members of your group about what disease traits they might be interested in that may be influenced by genetic background. Go to the OMIM (Online Mendelian Inheritance in Man) database (you can also get to it from the NCBI home page, on the drop-down menu under "All Databases". and research those genes of interest to your group. You should come up with at least one gene per group member – more would be better.

As an example, a search of "ceroid-lipofuscinosis" turns up 47 hits in OMIM - let's suppose that I am interested in the gene(s) that are responsible for this disease. If you click on the 7th hit "#601780 - Ceroid Lipofuscinosis", this gives you lots of information on this well-studied gene - we are interested in its protein sequence.

Now go back to the previous page and click on "Gene summaries" under the #601780 - Ceroid Lipofuscinosis link. On this new page, under "External Links" on the right-hand side, scroll down and click on "Protein" – a "UniProt" hypertext link appears – click on this. The protein found in this database is Ceroid-lipofuscinosis neuronal protein 6, encoded by the Gene CLN6.

Scroll down to "Sequences 2+" and "Isoform 1", and click on the "FASTA" button to the right. This gives you the primary amino acid sequence of this protein in a format that you can copy and save, and then use in a BLAST search.

Better yet, you can even run BLAST straight from the top of the page – just click on "BLAST" just underneath the Title. in the right-hand column. Now click on the blue "BLAST" button - it will take a minute or two to perform the alignment. When the BLAST is performed for this protein, a large number of significant "hits" are obtained, not surprisingly they are mostly from fellow primates.

To get a greater number of "hits" (the default is 25 for the Uniprot-based BLAST), use the drop-down menu and set it to 250 and rerun the BLAST on Uniprot. In this example, I would click on "B1HOZO" (near the bottom of the list of hits), because I am curious to know what protein in toads (*Xenopus tropicalis*) is similar to this human disease protein. It has a 70.7% identity with its human counterpart (orthologue).

Once you have chosen YOUR potential (human) sequences, use each of them as a query in BLASTp (protein-protein) to see what other proteins in the database have similarity to it. Pick one of the hits as the second protein sequence that will act as the second input to your program.

**DO NOT** pick a second protein that has extremely strong similarity to your first protein. **% Identity should be 65% or less** - this is listed in the third column from the right in the list of alignments from NCBI BLAST. **The protein should also be of sufficient length - pick proteins no shorter than 200 amino acids.** Picking a more distant protein relative will ensure a much better test of your program versus BLAST.

Copy and paste the two protein sequences (your human one, and the hit you picked from the BLAST results) into your program to see if your program can align them too! (You'll have to massage the second protein a bit after pasting since you'll have to get rid of gaps (dashes) that the BLAST result shows.)

Did your own program align the two sequences as well as BLAST did? Maybe, maybe not. It's hard to tell just by looking. You'll get the computer to help you answer that question in the next part below.

*Please keep google doc copies of the protein sequences you picked (including addresses of the web pages where you got them) and turn them in with your assignment. We want to test them too!*

## Is this biologically sound?

Getting your program to work correctly is only half the battle. Just because it works does not mean it is doing something useful to a biologist. To check, you will compare the results from your program with results obtained from the real BLAST. The **biology student(s)** on your team are responsible for writing the following code:

In a separate Python file named `nw_blast.py`, write two functions that will help analyze the usefulness of the output from your alignment matcher. The two functions are as follows:

```
1 def countMatches(query, subject):
2     # Takes the two parts of the alignment
3     # (the output from the first program)
4     # and returns the number of exact matches
5     # between them.
6     #
7     # For example, "PYSRSDD" and "P--VSDD"
8     # will return four. You may assume the two
9     # parameter strings have equal length.
```

```
1 def longest_consecutive(query, subject):
2     # Returns the length of the longest run
3     # of consecutive matches between the two
4     # parameters.
5     #
6     # For example, "PYSRSDD" and "P--VSDD"
7     # would return three, since the longest run of consecutive,
8     # exact matches was the last three amino acids: "SDD"
```

Once working, use the BLAST results that you found for test data above and compare those results to what your alignment program gives. Which is better? For example, one alignment BLAST might give when searching to align the EMSY protein may be this:

```
MPVVWPTLLDLS-RDECKR---ILRKLEL
--IMWPR-LLDME---EKECRSVLRRL--
```

where the top one is the query and the bottom one the subject. On the other hand, inputting these two strings (without the gaps) into the alignment matcher you just wrote may give an alignment of:

```
---MPVVWPTLLDLS-RDECKR-ILRKLEL
--IMWPRLLDM----EEKECRSVLRRL---
```

Run the two functions on each of the results to see (1) which algorithm (BLAST or yours) is delivering the most exact matches and (2) which algorithm is delivering the longest consecutive matches. Do the same on the human protein examples your group researched. Once you've collected some data, **answer the following questions in a separate write-up that you will turn in with your code:**

1. Is your program doing a global or a local alignment?
2. Based on your answer to the previous question, explain why you got the test results you got.
3. In what ways could you modify the alignment matcher you wrote to be more biologically sound?

## How to turn this in

Please turn in all Python files, including `nw_blast.py`, and the write-up named `nw_blast.pdf` on grade-scope. Don't forget to put the honor code affirmation in the comments of both.

|                         | Exceeding expectations (1x)                                                      | Arriving at expectations (1.5x)                                                  | Not meeting Expectations (0.5x)                                                    |
|-------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| CS elements<br>(6 pts)  | Correct implementation of N-W algorithm                                          | Correct logic of N-W algorithm but with semantic error                           | Incorrect logic of N-W algorithm                                                   |
|                         | Correct scoring and path matrices (including boundary conditions)                | Scoring and path matrices incorrect for boundary conditions                      | Incorrect scoring and path matrices                                                |
|                         | Correct <code>countMatches</code> and <code>longest_consecutive</code> functions | Correct <code>countMatches</code> xor <code>longest_consecutive</code> functions | Incorrect <code>countMatches</code> and <code>longest_consecutive</code> functions |
| Bio elements<br>(6 pts) | Appropriate test data                                                            | Some appropriate test data                                                       | Insufficient appropriate test data                                                 |
|                         | Conclusions regarding biological soundness                                       | Conclusions partially regarding biological soundness                             | Conclusions not regarding biological soundness                                     |
| Reflective<br>(8 pts)   | Thorough and understandable write-up of results                                  | Thorough xor understandable write-up of results                                  | Neither thorough nor understandable write-up of results                            |
|                         | Thorough and reflective write-up of team dynamics (peer review)                  | Thorough xor reflective write-up of team dynamics (peer review)                  | Neither thorough nor reflective peer reviews                                       |